

Autoencoder-Based Anomaly Detection in Network Traffic

Krzysztof Korniszek
Faculty of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland
korniszek.krzysztof@gmail.com

Bartosz Sawicki
Faculty of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland
bartosz.sawicki@pw.edu.pl

Abstract—Due to the continuously increasing number of resources and data availability in the cloud, the threats related to the security of computer networks and IT systems are critical. Threat detection systems based on deep neural networks and anomaly detection are trained on data related to normal activity so that the network can recognize unusual patterns and behaviours in the event of an attack or an attempt to infiltrate a given IT infrastructure. This paper presents the results of developing a neural network based on an autoencoder for anomaly detection in network packet data. The network was trained on data from the HIKARI-2021 dataset. The autoencoder aims to learn representations of normal network traffic and associate this type of traffic with a minimal reconstruction error. The obtained results were compared with those achieved by authors of other works. High accuracy and sensitivity were achieved at the cost of rather low precision, resulting in many false-positive results. A simple algorithm based on a single threshold value proved efficient but limited in terms of effectiveness. This problem can be resolved by changing the method of calculating the individual components of the vector, using only a subset of features, and deriving multiple vectors, one for each class separately, which has been described and analyzed in more detail.

Index Terms—anomaly detection, deep learning, cybersecurity, autoencoder, threat detection

I. INTRODUCTION

The security of IT systems has been a critical concern since their initial commercial deployment. With constantly evolving environments and the emergence of new types of cyberattacks, relying solely on security systems based on predefined behavioural patterns is no longer feasible. One promising solution to this challenge is the application of machine learning (ML) and artificial intelligence (AI) techniques in cybersecurity. Security vulnerabilities can lead to the loss of data, financial assets, and reputational damage, which can be difficult to recover from. To mitigate these risks, machine learning approaches have been increasingly employed, as they can process large datasets using sophisticated algorithms.

Machine learning techniques can be categorized into three primary types: supervised, unsupervised, and reinforcement. One of the significant advantages of ML is its ability to rapidly and effectively analyze vast quantities of data—a necessity in cybersecurity, where IT systems generate massive volumes of logs, events, and entries.

A key area within machine learning is deep learning, a subfield that mimics the human learning process. Deep

learning models, particularly neural networks, can process large datasets, which is essential for cybersecurity applications. Examples of AI-based systems for threat detection in IT networks include security information and event management (SIEM) platforms and intrusion detection systems (IDS). By leveraging deep learning, these systems can uncover complex patterns in network traffic, such as port activity or packet header details, which may be imperceptible to human analysts. This capability to detect subtle correlations in network data significantly enhances the effectiveness of anomaly detection in computer network security.

One technique worth mentioning in the context of security is anomaly detection [1]. Machine learning algorithms, both traditional and deep learning, can be trained to recognize normal activity in a given system. Then, when given a sample of data that stands out, the algorithm should be able to tell the difference. These techniques enable the detection of subtle deviations from normal network behaviour, which may indicate a range of security threats, including Distributed Denial of Service (DDoS) attacks, unauthorized access, or data exfiltration.

In this work, publicly available dataset HIKARI-2021 will be briefly analyzed. Then, a deep learning-based autoencoder algorithm will be presented, followed by a comparison with other approaches applied to the same dataset and their respective results. Finally, potential improvements and directions for future research will be proposed.

II. DATASET

The OSI (Open Systems Interconnection) model plays a critical role in IT security. It defines communication between the various layers of an IT system, with data being transmitted from the application layer—closest to the user—through the transport layers, down to the physical layer. This model is crucial for security as threats can be detected at different stages of communication. In the application layer, for example, threats such as malicious queries to a server or attempts to steal user credentials can be identified. The datasets analyzed in this work primarily focus on the network layer.

Several publicly available datasets were examined for this study. Despite the rapid evolution of cybersecurity, many studies still rely on outdated datasets, such as the KDD99

dataset, which is 25 years old at the time of this writing. Given the constant changes in the cybersecurity landscape, utilizing more up-to-date datasets that reflect current standards and attack patterns would likely yield more accurate and meaningful research outcomes.

The HIKARI dataset was selected for further analysis due to its size, number of features, and focus on encrypted data. It is also the most recent dataset among those analyzed in this paper, released in August 2021. The dataset was generated in a controlled laboratory environment, simulating real-world user behaviour and incorporating synthetic attack scenarios [5]. It contains over 80 features and approximately 250,000 records, each labelled to indicate whether it represents an attack. The HIKARI dataset includes three types of attacks: XMRIGCC, CryptoMiner, Bruteforce-XML, and Bruteforce.

Most of the HIKARI features come from a similar CICIDS-2017 dataset. They include information on basic flow features (duration, IP addresses), packet lengths, and statistics such as mean and standard deviation for packet and header lengths. Additionally, source and destination IP addresses and ports are provided [5].

III. IMPLEMENTATION OF ANOMALY DETECTION ALGORITHM

One type of network that can be used for anomaly detection tasks is an autoencoder. This is a specific type of neural network whose input and output size are the same. The main task of an autoencoder is to reconstruct the input data. It consists of two parts - encoder and decoder.

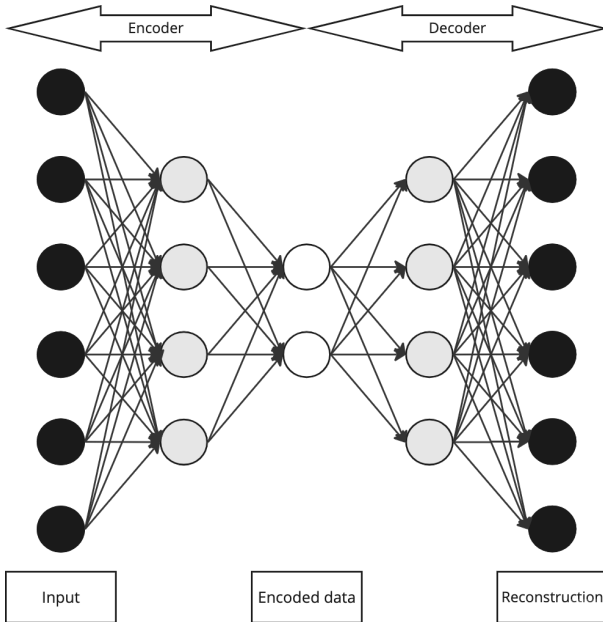


Fig. 1. The structure of autoencoder. Input and output layers have more neurons than the hidden layers.

The learning process assumes a reduced number of neurons in hidden layers compared to input layers, which will allow for the reconstruction of the input feature pattern without distortions, as shown in Figure 1. The hidden layer is tasked with

collecting the most important features of the set in a reduced form, presented using a smaller number of dimensions. The previously mentioned encoder learns the feature representation by using narrowed deep layers - according to the manifold hypothesis, high-dimensional data can be represented with fewer dimensions [6]. The decoder is responsible for creating a reconstruction of input features using the encoded information in the hidden layers. Figure 1 presents an example of an autoencoder with three hidden layers. For simplicity of mathematical description, we can reduce the problem to only one layer. Then, an encoder can be defined as the following expression:

$$H = \sigma(W_x X + b_x), \quad (1)$$

where H is the representation of data in the hidden layer, σ is activation function, W_x is the weight matrix, X is an input vector and b is the bias vector [8]. It transforms the input vector X to the hidden representation H using the activation function σ . The transformation on the hidden layer H aims to reconstruct the input layer using the decoder:

$$\hat{X} = \sigma(W_x H + b_{\hat{x}}), \quad (2)$$

which implies a reconstructed vector $\hat{X}$ of X . The difference between the reconstructed vector $\hat{X}$ and X produces a reconstruction error:

$$r = \|X - \hat{X}\| \quad (3)$$

The use of autoencoders to reduce data complexity is useful in the context of cybersecurity. The encoder reduces the number of dimensions in the input data. In anomaly detection, the data for training the network must represent only normal activity in the system. Then, normal and anomalous data can be found in the test set. The second part of the network, the decoder, aims to reconstruct the input data. In analysis detection, the previously mentioned concept of reconstruction error is used. It can be assumed that a high value of the reconstruction error indicates anomalous data and a low value indicates normal data.

A separate issue is determining the exact value of reconstruction error threshold above which the data will be treated as anomalies. The Receiver Operating Characteristic Curve (ROC) was the method used to develop the anomaly detection mechanism. It is a graph showing the performance of a binary classifier, in the case of indicating whether a given sample is an attack (see Fig. 3). The ROC curve is plotted on a unit square, where the points correspond to the sensitivity and precision values at different cut-off thresholds. The x-axis shows the percentage of false positives among all true negative cases. The y-axis shows the sensitivity, i.e., the percentage of true positives among all true positive cases. The better the model, the larger the area under the graph. The whole thing aims at compromising between high precision and sensitivity [2]. The goal is to select such a reconstruction error value to achieve

the lowest possible number of false positive samples while maintaining the highest possible number of true positives.

As part of the experiment, a neural network based on the architecture from Table I was implemented. This configuration comes from the work by Catillo et al [3]. The experiment also tested other configurations from the mentioned work, but this one showed the best results.

TABLE I
USED NEURAL NETWORK ARCHITECTURE

Layer type	Activation function	N. of dimensions
Input	-	82
Hidden 1	tanh	16
Hidden 2	sigmoid	4
Hidden 3	sigmoid	16
Output	tanh	82

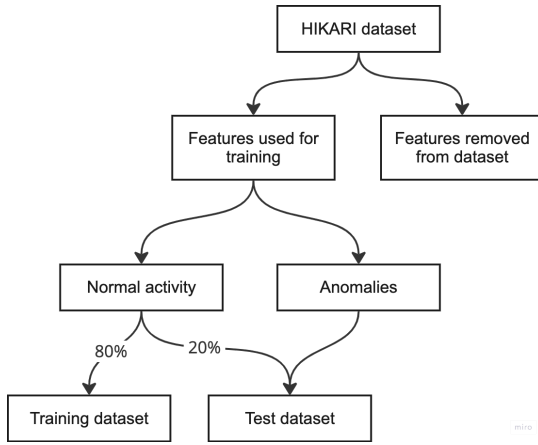


Fig. 2. Division of the HIKARI dataset into training and test data

The dataset was divided into normal and attack data. Figure 2 shows the division of HIKARI dataset for further training. Redundant features, including IP addresses and possible identifiers, were removed from it. 80% of the normal data was used for training, whereas data with attacks and the remaining 20% of the normal data was used for testing, as shown in Figure 2. The model training was performed on the resulting data set - 10% of the data was used for validation, the batch size was 256 units, and the number of epochs was equal to 30. An attempt was also made to select other values for the given hyperparameters, but the default ones turned out to be the best - the result of this research is shown in Table II.

TABLE II
THE IMPACT OF HYPERPARAMETERS CHANGES ON THE RESULTS OF THE NEURAL NETWORK.

Param	Default	Batch size		Epochs	
		128	512	20	40
Accuracy	94%	94%	94%	94%	84%
Precision	81%	81%	81%	81%	60%
Recall	99%	99%	99%	99%	99%
F1	89%	89%	89%	89%	74%

The default configuration refers to 30 epochs and 256 units as a batch size. Standard metrics were used for evaluation. Keras, Numpy and Pandas libraries were used to implement the algorithm. Additionally, tools for calculating model statistics were used from the Sklearn machine learning package. The code was implemented in the Jupyter Notebook environment. The model was trained on a MacBook Pro with a six-core Intel Core i9 processor and 32 GB of RAM.

IV. OBTAINED RESULTS

For the trained model, it is crucial to determine the reconstruction error threshold. For this purpose, the ROC method mentioned in the previous section of this paper was used. The curve is visible in Figure 3. It represents the best possible configuration obtained with the trained model, which was exactly equal to 0.042.

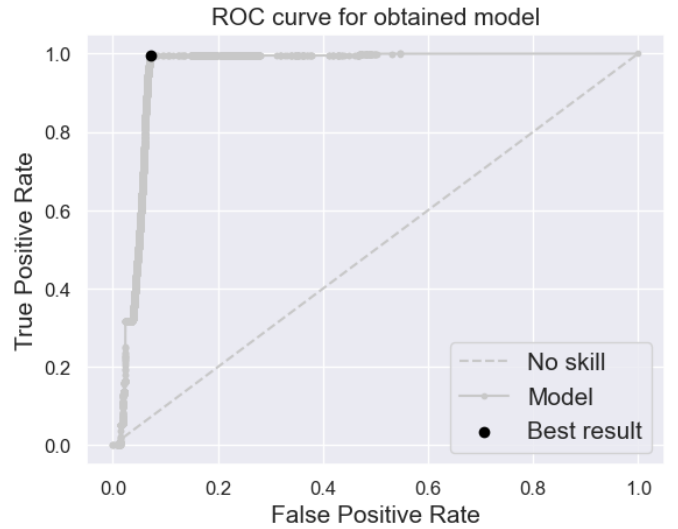


Fig. 3. ROC curve - the line at the top of the graph represents the model and the black point is the best possible configuration.

The reconstruction error threshold is shown in Figure 4. Points representing the average values of the reconstruction error for individual points are shown for normal data as brighter points and for anomalies as darker ones. Anything above the reconstruction error threshold, shown as a dark dashed line, should be an anomaly, and anything below it should be normal activity. As can be seen, not all points were correctly reconstructed by the decoder.

Table III shows the results obtained from our method and compares them with other authors works. It contains comparisons with algorithms from the works of Fernandes et al. [4] and Vitorino et al. [9]. These include implementations for random forest, extreme gradient boosting, KNN, and MLP.

It can be seen that our algorithm had a high level of recall and accuracy. A problem was the low precision parameter, which was about 80%. Figure 4 shows a single value of the reconstruction error threshold as the average value of the individual data components. In this figure, we can see that most of the points with anomalies are above the line representing

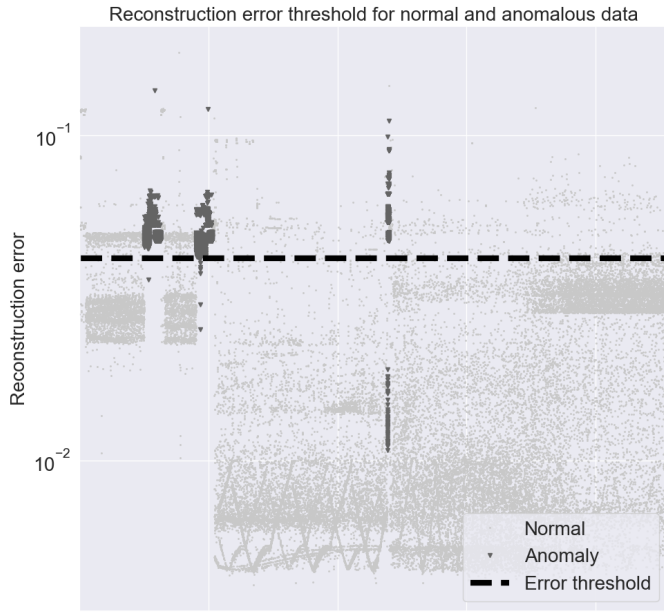


Fig. 4. Bright dots represent normal data, while dark represent anomalies. The dashed line is a representation of the reconstruction error threshold.

TABLE III
COMPARISON WITH THE RESULTS FROM RELATED WORKS.

Parameter	Our	Fernandes		Vitorino	
		RF	XGB	KNN	MLP
Accuracy	94%	98%	96%	98%	90%
Precision	81%	99%	99%	98%	90%
Recall	99%	69%	44%	98%	90%
F1	89%	81%	61%	98%	89%

the reconstruction error threshold. On the other hand, there are also a large number of points with normal data, which means false positive alerts. An attempt was also made using the method from the work of Torabi [8], but it gave poor results, below 80% accuracy. This results from the imperfections of the data in the training set - small deviations in the values for each of the features are enough and it generates significantly overstated thresholds, which are then required to classify the data as anomalies. It is worth mentioning that the authors of the aforementioned work developed a method based on many classes. Our algorithm would need to be refined for this purpose.

V. CONCLUSIONS, FURTHER WORK AND SUMMARY

The developed algorithm did well detecting threats and could detect most attacks from the test set. The problem is the precision parameter, which at a value of about 80% means that the solution is characterized by a high number of false positives. In the case of cybersecurity, it poses a problem due to the costs and time required to analyze such reports. The reason is identified as a single value of the reconstruction error threshold. Further research is planned to improve the algorithm's performance. The following methods of performance improvement are proposed:

- Using multiple reconstruction error threshold vectors instead of just one. For each of the individual classes in a set, a reconstruction error threshold vector can be determined, which should improve the classification of individual threats. With a single error threshold value, a large number of false positive events occur. Studies have shown that a single reconstruction error threshold vector deviates significantly from the test data. Due to its high value for individual components, practically all samples are classified as normal activity. A single discrepancy in the data is enough and the vector value takes on very high values relative to the set.
- Classification based on only some of the features. Not all features will have the same impact on whether a given sample is an anomaly. Using the chi-square method to analyze individual features and using them in the data classification process should yield better results in threat detection and reduce the number of false positives.
- Change of the method of calculating components of the error threshold vector. In this paper, it was chosen for each component as the maximum value of all input records for the corresponding feature. A solution to this may be to calculate the mean for all data and then the standard deviation.

REFERENCES

- [1] Ahmed M., Mahmood A. N., Hu J. "A survey of network anomaly detection techniques", *Journal of Network and Computer Applications*, 2016, 60: 19-31.
- [2] Bradley A. P., "The use of the area under the ROC curve in the evaluation of machine learning algorithms," *Pattern Recognition*, 1997.
- [3] Catillo M., Pecchia A. and Villano U., "AutoLog: Anomaly Detection by Deep Autoencoding of System Logs," *Expert Systems with Applications*, 2021.
- [4] Fernandes R., Silva J., Ribeiro Ó., Portela I. and Lopes N., "The impact of identifiable features in ML Classification algorithms with the HIKARI-2021 Dataset," *IEEE*, 2023.
- [5] Ferriyan A., Thamrin A. H., Takeda K., and Murai J., "Generating network intrusion detection dataset based on real and encrypted synthetic attack traffic," *IEEE*, 2021.
- [6] Goodfellow I., Bengio Y. and Courville A., "Deep Learning," *The MIT Press*, 2016.
- [7] Holly S. "Autoencoder based Anomaly Detection and Explained Fault Localization in Industrial Cooling Systems," *Cornell University*, 2022.
- [8] Torabi H., Mirtaheeri S. L. and Greco S., "Practical autoencoder based anomaly detection by using vector reconstruction error," *Springer Open*, 2023.
- [9] Vitorino J., Silva M., Maia E., Praça I., "Reliable Feature Selection for Adversarially Robust Cyber-Attack Detection," *Cryptography and Security*, 2024.